

Práctica 1 ITBD: Ingesta y Almacenamiento de Datos

Nicolás Vives Vicente

Universidad de Alicante
Máster en Ciencia de Datos

Septiembre 2025

Fase 1: Ingesta Local

Paso 1: Instalación y configuración

Seguimos los pasos descritos en el enunciado de la práctica para descargar e instalar **Node.js** y **npm**, el entorno de ejecución de Node-RED.

Una vez instalados desde la terminal podemos instalar Node-RED mediante el comando: `npm install -g -unsafe-perm node-red`.

Seguimos los pasos indicados para instalar mosquitto y lo iniciamos desde la terminal mediante el comando: `mosquitto -p 8883 -v` (para usar el puerto 8883). Este comando debe ser ejecutado en la ruta en la que se encuentre el archivo mosquitto.exe.

Una vez abierto Node-RED en el puerto `http://127.0.0.1:1880/`, instalamos la librería Cheerio desde el menú `>Manage Palette >Install` y buscar cheerio.

Paso 2: Creación de los Flujos en Node-RED

Flujo 1: Simulación de Consumo del Hogar

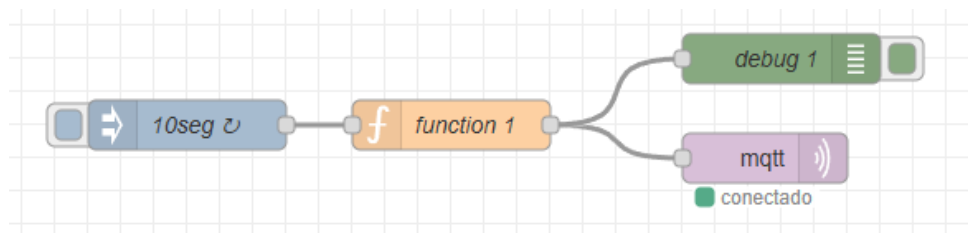


Figura 1: Flujo 1. Simulación de Consumo del Hogar

1. Nodo inject se repite cada 10 segundos

The image shows a configuration window for a node named "10seg". At the top, there's a label "Nombre" with the value "10seg". Below it, there are two configuration rows. The first row is for "msg. payload" set to "milisegundos desde epoch". The second row is for "msg. topic" set to "a_z". Below these rows are two buttons: "+ añadir" and "inyectar ahora". Further down, there's a checkbox labeled "Inyectar una vez después de 0.1 segundos, entonces", which is currently unchecked. Below that, there's a "Repetir" section with a dropdown menu set to "intervalo". At the bottom, there's a "cada" field set to "10" and a unit dropdown set to "segundos".

Figura 2: Configuración del nodo inject cada 10 segundos

2. El nodo function está configurado de la siguiente manera

```
const numHouses = 5; // Simula 5 hogares
const messages = [];
const now = new Date();
const hour = now.getHours();
// Patrón de consumo simulado por hora
let baseConsumption;
if (hour >= 0 && hour < 7) {
  baseConsumption = Math.random() * 0.6 + 0.2;
} else if (hour >= 7 && hour < 21) {
  baseConsumption = Math.random() * 0.7 + 0.8;
} else {
  baseConsumption = Math.random() * 1.0 + 1.5;
}
for (let i = 1; i <= numHouses; i++) {
  const consumption = baseConsumption + (Math.random() - 0.5) * 0.5;
  messages.push({
    topic: 'energo/consumo/energo_house_${i.toString().padStart(2, '0')}',
    payload: {
      consumption_kw: parseFloat(consumption.toFixed(2)),
      timestamp: now.toISOString(),
      unit: "kW"
    }
  });
}
return [messages];
```

3. Con el módulo debug 1 vemos que estamos obteniendo el output correcto

```
18/9/2025, 17:05:51 nodo: debug 1
energo/consumo/energo_house_05 : msg.payload : Object
  ▶ { consumption_kw: 1.18, timestamp: "2025-09-18T15:05:51.051Z", unit: "kW" }

18/9/2025, 17:05:56 nodo: debug 1
energo/consumo/energo_house_01 : msg.payload : Object
  ▶ { consumption_kw: 0.9, timestamp: "2025-09-18T15:05:56.181Z", unit: "kW" }

18/9/2025, 17:05:56 nodo: debug 1
energo/consumo/energo_house_02 : msg.payload : Object
  ▶ { consumption_kw: 0.94, timestamp: "2025-09-18T15:05:56.181Z", unit: "kW" }

18/9/2025, 17:05:56 nodo: debug 1
energo/consumo/energo_house_03 : msg.payload : Object
  ▶ { consumption_kw: 0.9, timestamp: "2025-09-18T15:05:56.181Z", unit: "kW" }

18/9/2025, 17:05:56 nodo: debug 1
energo/consumo/energo_house_04 : msg.payload : Object
  ▶ { consumption_kw: 0.99, timestamp: "2025-09-18T15:05:56.181Z", unit: "kW" }

18/9/2025, 17:05:56 nodo: debug 1
energo/consumo/energo_house_05 : msg.payload : Object
  ▶ { consumption_kw: 0.78, timestamp: "2025-09-18T15:05:56.181Z", unit: "kW" }
```

Figura 3: Output del módulo debug 1

4. En la configuración del mqtt out dejamos el topic vacío ya que este viene dado por el nodo function tal y como lo hemos configurado.

```
topic: 'energo/consumo/energo_house_${i.toString().padStart(2, '0')}'
```

The image shows the configuration interface for the MQTT Out module. It includes the following fields and settings:

- Servidor:** A dropdown menu showing 'localhost:8883' with edit and add icons.
- Tema:** A text input field that is currently empty.
- CdS:** A dropdown menu with a downward arrow.
- Retener:** A checkbox that is checked, with a label 'Retener'.
- Nombre:** A text input field that is currently empty.

Figura 4: Configuración del módulo mqtt out del Flujo 1

Flujo 2.1: Web Scrapping. Precio de la luz

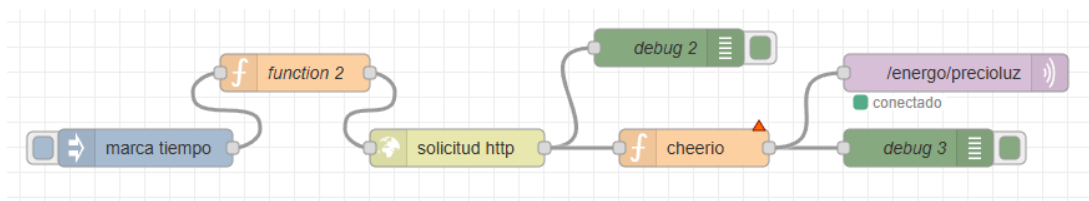


Figura 5: Flujo 2.1. Precio de la luz

1. Nodo inject para iniciar el flujo.
2. El módulo function 2 donde se parametriza la URL



Figura 6: Configuración del módulo function 2

3. El módulo HTTP Request tiene el campo de URL vacío ya que recibe como entrada la URL proporcionada por el módulo function 2.
4. El módulo cheerio contiene el código siguiente, este localiza el precio medio y lo devuelve en un mensaje junto con el año, mes y día actual.

```

let precio = $('strong:contains("Precio medio")')
    .parent()
    .parent()
    .next()
    .text();

precio = precio.substring(0, precio.length - 6);
precio = parseFloat(precio);

// Obtener fecha actual
let hoy = new Date();
let año = hoy.getFullYear();
let mes = hoy.getMonth() + 1; // empieza en 0, por eso sumamos 1
let dia = hoy.getDate();

```

```
// Construir el payload
msg.payload = {
  precio: precio,
  dia: dia,
  mes: mes,
  año: año
};

return msg;
```

5. Con el módulo debug 3 vemos que estamos obteniendo el output correcto

```
18/9/2025, 16:12:24  nodo: debug 3
msg.payload : Object
  ▼ object
    precio: 0.1594
    dia: 18
    mes: 9
    año: 2025
```

Figura 7: Output del módulo debug 1

6. La configuración del mqtt out con topic /energo/precioluz

The image shows the configuration interface for the MQTT Out module. It includes the following fields and controls:

- Servidor:** A dropdown menu showing 'localhost:8883' with a checkmark icon and a plus icon to add more servers.
- Tema:** A text input field containing '/energo/precioluz'.
- CdS:** A dropdown menu that is currently empty.
- Retener:** A checkbox that is checked, with a dropdown menu to its right.
- Nombre:** A text input field containing the word 'Nombre'.

Figura 8: Configuración del módulo mqtt out del Flujo 2.1

Flujo 2.2: Web Scrapping. Temperatura por día y hora

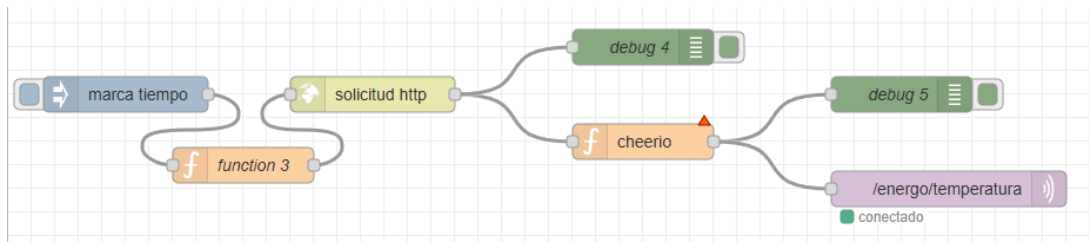


Figura 9: Flujo 2.2. Temperatura por día y hora

1. Nodo inject para iniciar el flujo.
2. El módulo function 3 donde se crean las distintas URL (una por día) contiene el siguiente código.

```
const anyo=2025, mes=8, diaInicio=1, diaFin=2;
const urls=[];
for (let d =diaInicio; d<=diaFin;d++) {
  urls.push({
    url:"https://www.meteociel.fr/temps-reel/obs_villes.php?code2=8359&jour2="+
    d+"&mois2="+mes+"&annee2="+anyo,
    anyo,
    mes,
    dia: d
  });
}
console.log(urls);
return [urls];
```

3. El módulo HTTP Request tiene el campo de URL vacío ya que recibe como entrada las URL proporcionadas por el módulo funcion 3.
4. El módulo cheerio contiene el siguiente código, este localiza las temperaturas por cada hora y retorna un array con un array de mensajes. Cada mensaje tiene en su payload: año, mes, día, hora y temperatura.

```
let filas = $("b:contains('Windchill')")
  .parent() // Subo al td
  .parent() // subo al tr
  .parent() // subo al tbody
```

```

        .children()

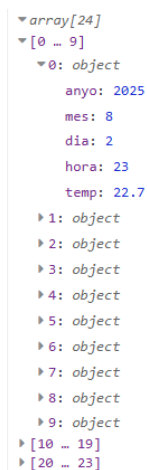
let valores = []; // array de valores vacío

filas.each(function() {
    let celdas = $(this).children();
    let hora = celdas.eq(0).text()
    if (hora!= "HeureUTC"){
        hora = hora.substring(0, hora.length-3);
        hora = parseInt(hora);
        let temp = celdas.eq(4).text();
        temp = temp.substring(0, temp.length-3);
        temp = parseFloat(temp);
        console.log("Hora:", hora, "Temp:", temp); // muestra las horas
            de la tabla en la consola
        valores.push({
            anyo: msg.anyo,
            mes: msg.mes,
            dia: msg.dia,
            hora: hora,
            temp, temp}); // añade valores al array
    }
});

msg.payload=valores
return msg;

```

5. Con el módulo debug 5 vemos que estamos obteniendo el output correcto



```

▼ array[24]
▼ [0 ... 9]
  ▼ 0: object
    anyo: 2025
    mes: 8
    dia: 2
    hora: 23
    temp: 22.7
  ▶ 1: object
  ▶ 2: object
  ▶ 3: object
  ▶ 4: object
  ▶ 5: object
  ▶ 6: object
  ▶ 7: object
  ▶ 8: object
  ▶ 9: object
  ▶ [10 ... 19]
  ▶ [20 ... 23]

```

Figura 10: Output del módulo debug 5

6. La configuración del mqtt out con topic /energo/temperatura

Servidor: localhost:8883
 Tema: /energo/temperatura
 CdS: Retener
 Nombre: Nombre

Figura 11: Configuración del módulo mqtt out del Flujo 2.2

Paso 3: Verificación de la Ingesta Local

Hemos ido comprobando a lo largo del documento mediante los módulos debug que para cada flujo los mensajes con el formato correcto aparecen en el panel de depuración.

En esta sección veremos que los mensajes publicados por los flujos llegan a mosquitto. Para ello hemos tenido que cambiar el puerto 8883 que se indica en la práctica por el 1884, ya que si no, no podíamos ejecutar correctamente `mosquitto_sub -h localhost -t # -v`, en una nueva terminal.

```

energo/consumo/energo_house_01 {"consumption_kw":1.11,"timestamp":"2025-10-01T08:44:13.291Z","unit":"kW"}
energo/consumo/energo_house_02 {"consumption_kw":1.04,"timestamp":"2025-10-01T08:44:13.291Z","unit":"kW"}
energo/consumo/energo_house_03 {"consumption_kw":0.96,"timestamp":"2025-10-01T08:44:13.291Z","unit":"kW"}
energo/consumo/energo_house_04 {"consumption_kw":1.24,"timestamp":"2025-10-01T08:44:13.291Z","unit":"kW"}
energo/consumo/energo_house_05 {"consumption_kw":1.29,"timestamp":"2025-10-01T08:44:13.291Z","unit":"kW"}
energo/consumo/energo_house_01 {"consumption_kw":1.37,"timestamp":"2025-10-01T08:44:23.299Z","unit":"kW"}
energo/consumo/energo_house_02 {"consumption_kw":1.18,"timestamp":"2025-10-01T08:44:23.299Z","unit":"kW"}
energo/consumo/energo_house_03 {"consumption_kw":1.42,"timestamp":"2025-10-01T08:44:23.299Z","unit":"kW"}
energo/consumo/energo_house_04 {"consumption_kw":1.33,"timestamp":"2025-10-01T08:44:23.299Z","unit":"kW"}
energo/consumo/energo_house_05 {"consumption_kw":1.22,"timestamp":"2025-10-01T08:44:23.299Z","unit":"kW"}
energo/consumo/energo_house_01 {"consumption_kw":1.31,"timestamp":"2025-10-01T08:44:33.304Z","unit":"kW"}
energo/consumo/energo_house_02 {"consumption_kw":1.17,"timestamp":"2025-10-01T08:44:33.304Z","unit":"kW"}
energo/consumo/energo_house_03 {"consumption_kw":1.11,"timestamp":"2025-10-01T08:44:33.304Z","unit":"kW"}
energo/consumo/energo_house_04 {"consumption_kw":1.11,"timestamp":"2025-10-01T08:44:33.304Z","unit":"kW"}
energo/consumo/energo_house_05 {"consumption_kw":1.26,"timestamp":"2025-10-01T08:44:33.304Z","unit":"kW"}
energo/consumo/energo_house_01 {"consumption_kw":0.76,"timestamp":"2025-10-01T08:44:43.312Z","unit":"kW"}
energo/consumo/energo_house_02 {"consumption_kw":0.97,"timestamp":"2025-10-01T08:44:43.312Z","unit":"kW"}
energo/consumo/energo_house_03 {"consumption_kw":1.04,"timestamp":"2025-10-01T08:44:43.312Z","unit":"kW"}
energo/consumo/energo_house_04 {"consumption_kw":0.79,"timestamp":"2025-10-01T08:44:43.312Z","unit":"kW"}
energo/consumo/energo_house_05 {"consumption_kw":0.68,"timestamp":"2025-10-01T08:44:43.312Z","unit":"kW"}
energo/consumo/energo_house_01 {"consumption_kw":1.19,"timestamp":"2025-10-01T08:44:53.313Z","unit":"kW"}
energo/consumo/energo_house_02 {"consumption_kw":1.46,"timestamp":"2025-10-01T08:44:53.313Z","unit":"kW"}
energo/consumo/energo_house_03 {"consumption_kw":1.26,"timestamp":"2025-10-01T08:44:53.313Z","unit":"kW"}
energo/consumo/energo_house_04 {"consumption_kw":1.37,"timestamp":"2025-10-01T08:44:53.313Z","unit":"kW"}
energo/consumo/energo_house_05 {"consumption_kw":1.38,"timestamp":"2025-10-01T08:44:53.313Z","unit":"kW"}
/energo/precipitacion {"precipitacion":10.1537,"dia":1,"mes":10,"año":2025}
/energo/temp [{"año":2025,"mes":8,"dia":2,"hora":19,"temp":25.8}, {"año":2025,"mes":8,"dia":2,"hora":18,"temp":26.5}, {"año":2025,"mes":8,"dia":2,"hora":17,"temp":27}, {"año":2025,"mes":8,"dia":2,"hora":16,"temp":28.7}, {"año":2025,"mes":8,"dia":2,"hora":15,"temp":28.6}, {"año":2025,"mes":8,"dia":2,"hora":14,"temp":29.2}, {"año":2025,"mes":8,"dia":2,"hora":13,"temp":29.5}, {"año":2025,"mes":8,"dia":2,"hora":12,"temp":28.0}, {"año":2025,"mes":8,"dia":2,"hora":11,"temp":30}, {"año":2025,"mes":8,"dia":2,"hora":10,"temp":30.8}, {"año":2025,"mes":8,"dia":2,"hora":9,"temp":29.6}, {"año":2025,"mes":8,"dia":2,"hora":8,"temp":27.1}, {"año":2025,"mes":8,"dia":2,"hora":7,"temp":23.1}, {"año":2025,"mes":8,"dia":2,"hora":6,"temp":20.6}, {"año":2025,"mes":8,"dia":2,"hora":5,"temp":21}, {"año":2025,"mes":8,"dia":2,"hora":4,"temp":22.1}, {"año":2025,"mes":8,"dia":2,"hora":3,"temp":22.9}, {"año":2025,"mes":8,"dia":2,"hora":2,"temp":23.8}, {"año":2025,"mes":8,"dia":2,"hora":1,"temp":23.9}, {"año":2025,"mes":8,"dia":2,"hora":0,"temp":22.9}], [{"año":2025,"mes":8,"dia":1,"hora":21,"temp":22.0}, {"año":2025,"mes":8,"dia":1,"hora":22,"temp":20}, {"año":2025,"mes":8,"dia":1,"hora":23,"temp":25.2}, {"año":2025,"mes":8,"dia":1,"hora":20,"temp":25.8}, {"año":2025,"mes":8,"dia":1,"hora":19,"temp":26.4}, {"año":2025,"mes":8,"dia":1,"hora":18,"temp":27.3}, {"año":2025,"mes":8,"dia":1,"hora":17,"temp":28.2}, {"año":2025,"mes":8,"dia":1,"hora":16,"temp":33.2}, {"año":2025,"mes":8,"dia":1,"hora":15,"temp":33.8}, {"año":2025,"mes":8,"dia":1,"hora":14,"temp":33.3}, {"año":2025,"mes":8,"dia":1,"hora":13,"temp":33.7}, {"año":2025,"mes":8,"dia":1,"hora":12,"temp":33}, {"año":2025,"mes":8,"dia":1,"hora":11,"temp":32.3}, {"año":2025,"mes":8,"dia":1,"hora":10,"temp":30.2}, {"año":2025,"mes":8,"dia":1,"hora":9,"temp":28.6}, {"año":2025,"mes":8,"dia":1,"hora":8,"temp":26.2}, {"año":2025,"mes":8,"dia":1,"hora":7,"temp":26.1}, {"año":2025,"mes":8,"dia":1,"hora":6,"temp":24.6}, {"año":2025,"mes":8,"dia":1,"hora":5,"temp":24.1}, {"año":2025,"mes":8,"dia":1,"hora":4,"temp":25.4}, {"año":2025,"mes":8,"dia":1,"hora":3,"temp":25}, {"año":2025,"mes":8,"dia":1,"hora":2,"temp":24.8}, {"año":2025,"mes":8,"dia":1,"hora":1,"temp":24.7}, {"año":2025,"mes":8,"dia":1,"hora":0,"temp":24.4}]

```

Figura 12: Terminal de mosquitto donde los mensajes llegan correctamente

Fase 2: Escalando a la Nube y Alta Disponibilidad

Ahora, migraremos el pipeline de ingesta a un entorno en la nube para gestionar la escala y la tolerancia a fallos.

Paso 1: Configuración en AWS

Creación de la cuenta de AWS

Creamos la cuenta con el email recibido y accedemos al laboratorio de la práctica.

Configuración de AWS IoT Core:

Tras marcar como favoritos IoT Core y S3 accedemos al menú de IoT Core para configurar un objeto. A continuación clicaremos en conectar dispositivo.

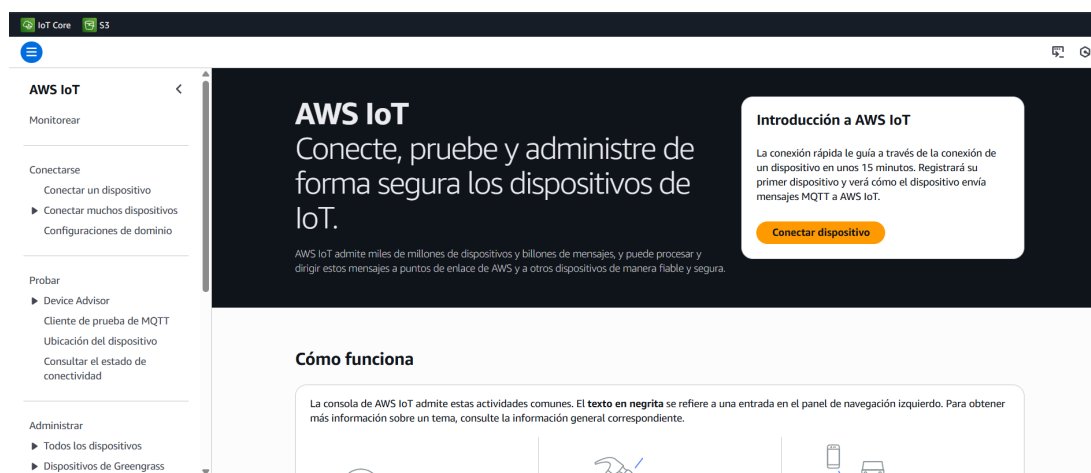


Figura 13: Menú de IoT Core para conectar un dispositivo

Seguimos los pasos que se muestran, llamamos al objeto: **objeto_energo_1_nico**.

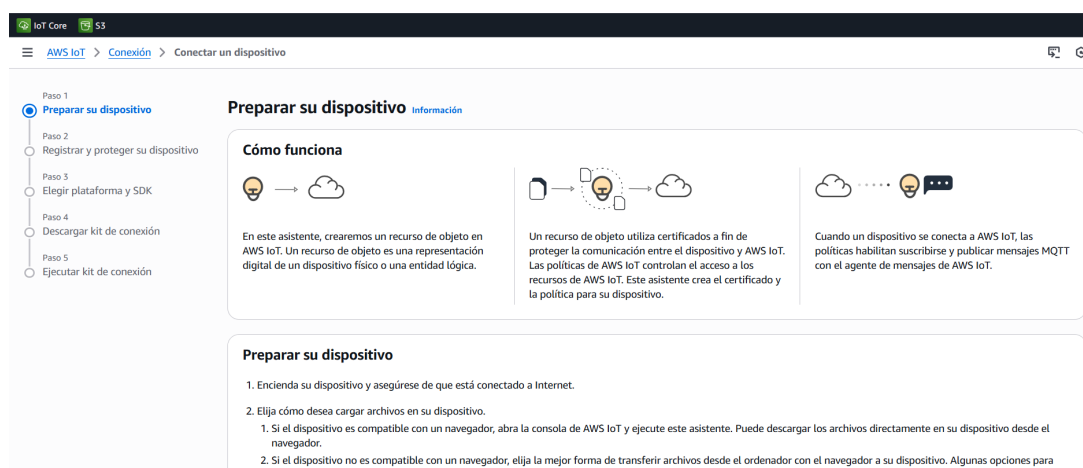


Figura 14: Menú de pasos para crear/preparar el dispositivo/objeto

En el paso 4 descargamos el kit de conexión del objeto, que contiene los siguientes archivos, excepto el Certificado Raíz, que es un archivo esencial para la seguridad y como el kit no lo incluye lo hemos descargado de <https://www.amazontrust.com/repository/AmazonRootCA1.pem>.

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
AmazonRootCA1.pem	✓	01/10/2025 11:18	Documento de tex...	2 KB
objeto_energo_1_nico.cert.pem	✓	01/10/2025 11:04	Archivo PEM	2 KB
objeto_energo_1_nico.private	✓	01/10/2025 11:04	Presentación de K...	2 KB
objeto_energo_1_nico.public	✓	01/10/2025 11:04	Presentación de K...	1 KB
objeto_energo_1_nico-Policy	✓	01/10/2025 11:04	Archivo	2 KB
start	✓	01/10/2025 11:04	Script de Windows...	2 KB

Figura 15: Contenido del kit de conexión del objeto

Finalmente le damos a crear y vemos que el objeto ha sido creado correctamente.

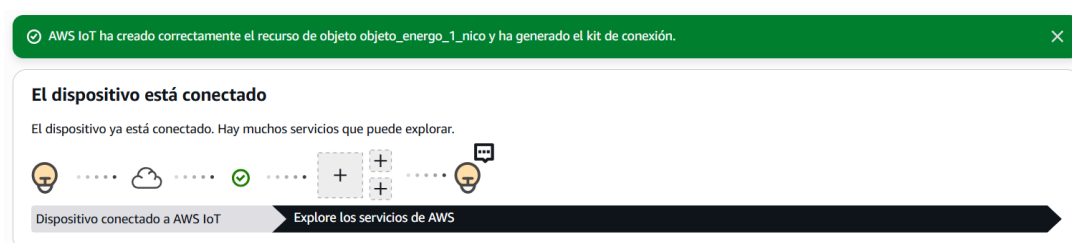


Figura 16: Verificación de creación del objeto

Paso 2: Configurar los Permisos en AWS

Como por defecto AWS crea una política que es muy restrictiva. Vamos a editarla para que el flujo de Node-RED pueda publicar en el topic.

Ir a la política

En la consola de AWS, vamos a **AWS IoT Core > Secure > Políticas**. Hacemos clic en la política con el nombre **objeto_energo_1_nico** y editamos estas políticas.



Figura 17: Interfaz de políticas en IoT Core

Editar el Documento de la Política

Reemplazamos el documento JSON de la política original por el siguiente que da permiso para conectar y publicar en cualquier tema.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Connect",
        "iot:Publish",
        "iot:Receive",
        "iot:Subscribe" ],
      "Resource": "*"
    }
  ]
}
```

Se ofrecieron políticas alternativas para aquellos alumnos que debido a estas no conseguían la conexión, sin embargo, al no ser este el caso, adoptamos las políticas recomendadas en el enunciado de la práctica.

Después en **IoT Core > Secure > Certificates**, hacemos clic en el certificado del dispositivo y comprobamos que la política está adjunta.

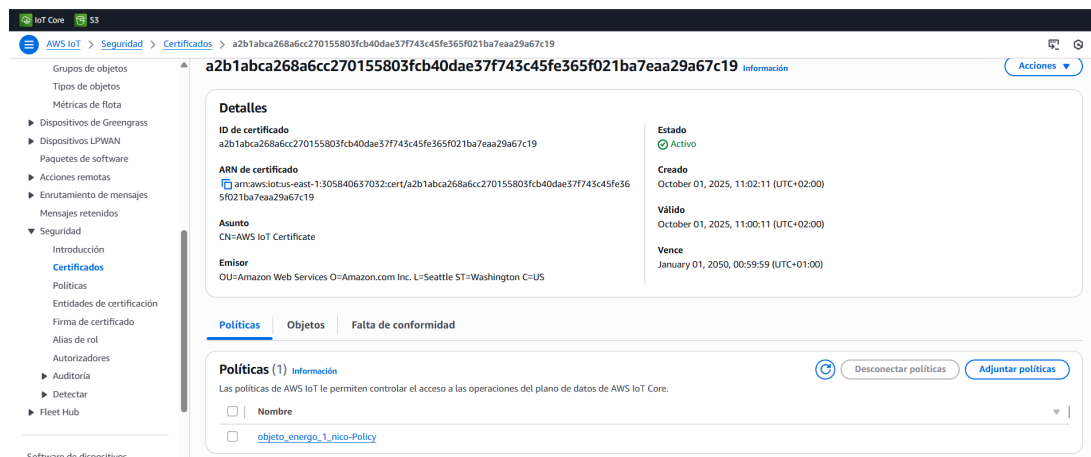


Figura 18: Política adjunta al dispositivo

Paso 3: Configurar el Flujo de Node-RED

Abrir el flujo

Abrimos Node-RED con nuestro flujo de simulación de consumo de nuevo ejecutando en la terminal `node-red` y dirigiéndonos a `http://127.0.0.1:1880/`.

Configurar el Nodo mqtt out

Duplicamos los nodos mqtt out para cada uno de los 3 flujos y configuramos las siguientes opciones:

- En “Server”, pegamos el endpoint que extraemos del script del kit. Usamos el puerto 8883 para la conexión segura.

Edit mqtt out node > **Edit mqtt-broker node**

Delete Cancel Update

Properties

Name

Connection Security Messages

Server a1s639ohtzloz-ats.iot.us-east-1.amazonaws.com Port 8883

☒ Connect automatically

☒ Use TLS TLS configuration

Protocol MQTT V3.1.1

Client ID Leave blank for auto generated

Keep Alive 60

Session ☒ Use clean session

Figura 19: Configuración del servidor de aws en el mqtt

- En la pestaña “TLS”, subimos los tres archivos de seguridad: el certificado del dispositivo, la clave privada y el certificado raíz de Amazon.

Edit mqtt out node > Edit mqtt-broker node > **Edit tls-config node**

Delete Cancel Update

Properties

☐ Use key and certificates from local files

Certificate Upload objeto_energo_1_nico.cert.p...

Private Key Upload objeto_energo_1_nico.privat...

Passphrase private key passphrase (optional)

CA Certificate Upload AmazonRootCA1.pem.txt

☒ Verify server certificate

Server Name for use with SNI

ALPN Protocol for use with ALPN

Figura 20: Archivos del kit subidos a la pestaña “TLS”

- Puesto que en las políticas hemos configurado el campo “Resource” como “*”, desde la perspectiva de permisos, cualquier topic puede ser autorizado para publicar, suscribir o recibir mensajes, ya que la política es muy permisiva. Esto implica que no hay limitación de topic por lo que lo configuramos como el nombre del objeto.

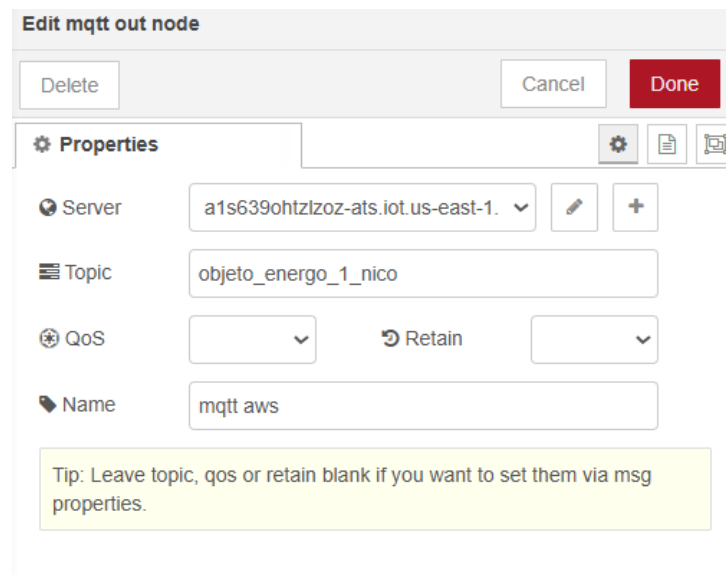


Figura 21: El nombre del topic coincide con el nombre del objeto

Paso 4: Pruebas y Verificación

Hacemos clic en “Deploy” en Node-RED y en la consola podemos comprobar que la conexión ha sido exitosa.

```
1 Oct 20:08:45 - [info] Stopped flows
1 Oct 20:08:45 - [info] Updated flows
1 Oct 20:08:45 - [info] Starting flows
1 Oct 20:08:45 - [info] Started flows
1 Oct 20:08:46 - [info] [mqtt-broker:8e8b016edb545a79] Connected to broker: mqtt://a1s639ohtzlzoz-ats.iot.us-east-1.amazonaws.com:8883
```

Figura 22: La conexión entre Node-RED y AWS ha sido exitosa

También podemos ver que los nodos mqtt out correspondientes a AWS están conectados directamente en Node-RED.

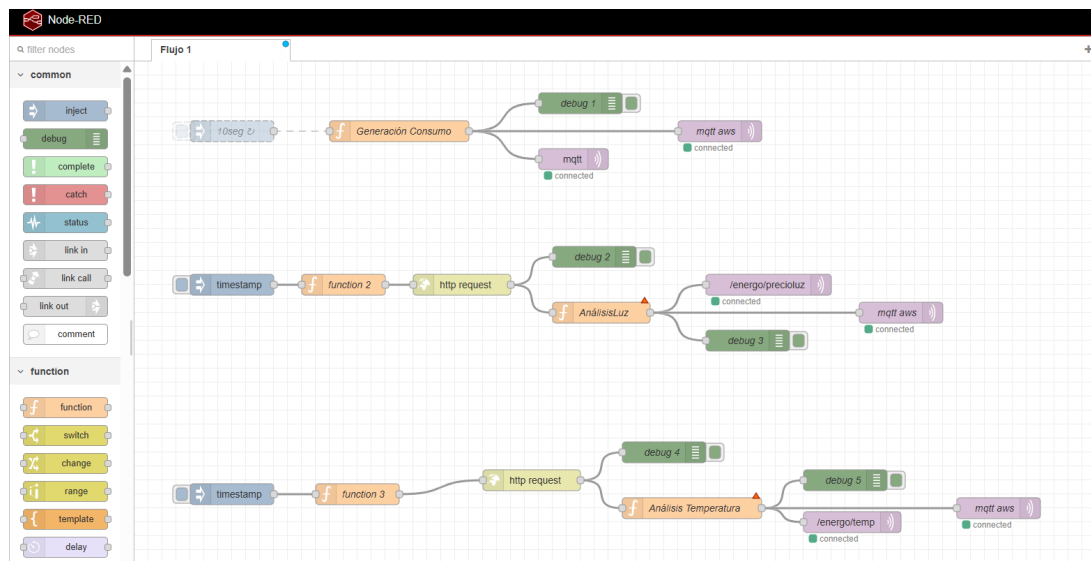


Figura 23: La nodos mqtt están conectados a AWS

Ahora procedemos a verificar los mensajes. En la consola de AWS vamos a **“IoT Core >Test >MQTT test client”** y nos suscribimos al topic **“objeto_energo_1_nico”**.

Una vez suscritos comprobamos que los mensajes de Node-RED aparecen en tiempo real, confirmando que el pipeline de ingesta funciona perfectamente.

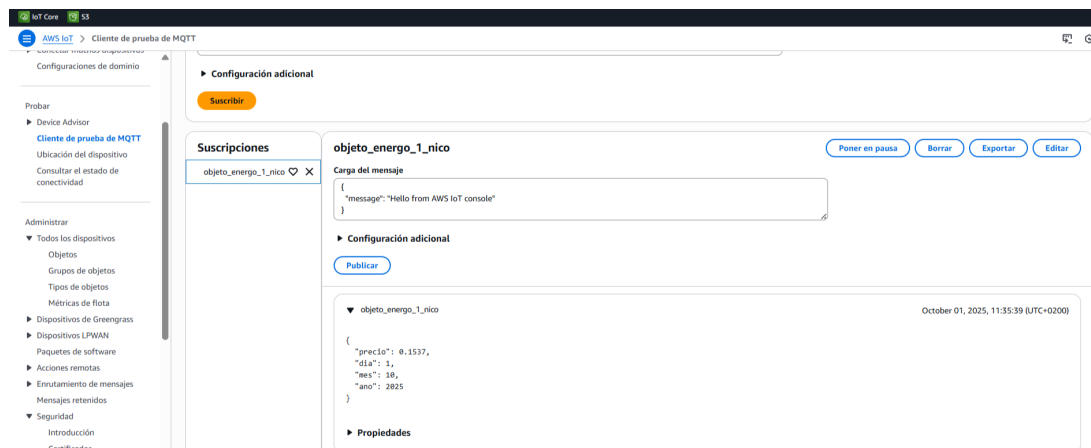


Figura 24: Suscripción al topic **“objeto_energo_1_nico”**

Paso 5: Almacenamiento

Ahora que ya tenemos la ingesta de información, vamos a utilizar el servicio S3 de AWS, estableciendo una regla que traspase la información del topic al Bucket de S3.

Crear el Bucket de S3

En la consola de AWS, buscamos “S3” y hacemos clic en “Crear bucket”. Se nos abre entonces la siguiente interfaz, donde hemos elegido el nombre del bucket “bucketenergolnico”.

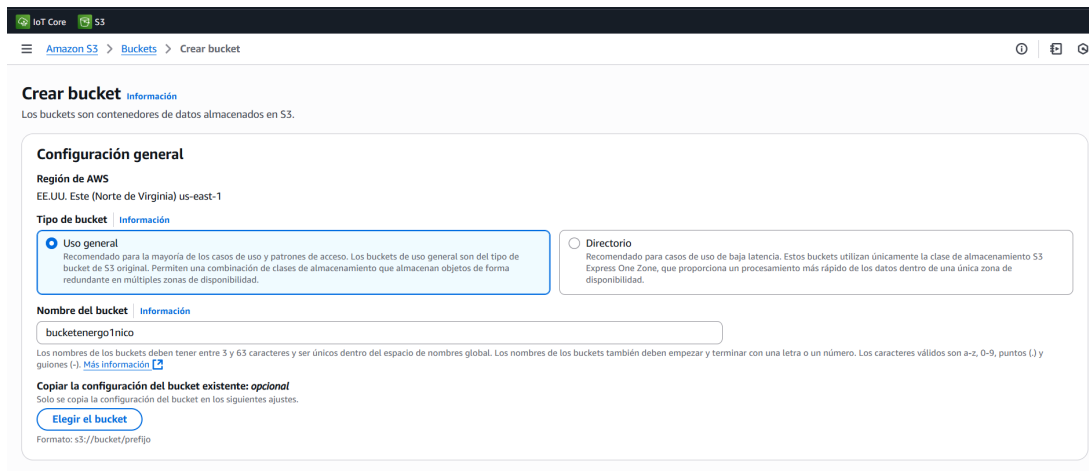


Figura 25: Interfaz de creación del bucket

Dejamos el resto de las opciones del bucket configuradas “por defecto” y hacemos click en “Crear bucket”

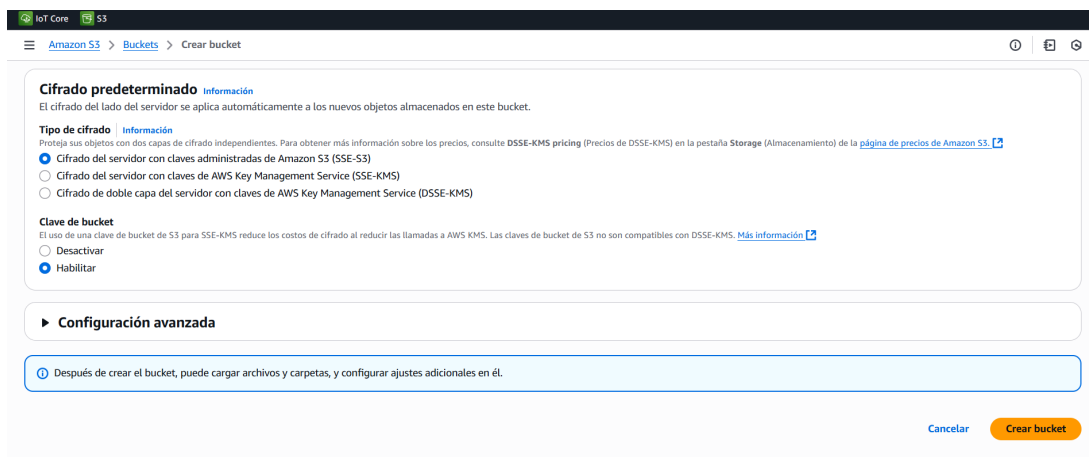


Figura 26: Interfaz de creación del bucket

Crear la Regla de Almacenamiento en IoT

Ahora volvemos a la consola de **AWS IoT Core**.

En el menú de la izquierda vamos a “**Administrar >Enrutamiento de mensajes >Reglas**”

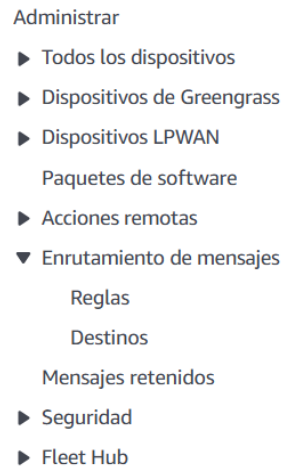


Figura 27: “Administrar >Enrutamiento de mensajes >Reglas”

Hacemos click en crear regla y le damos un nombre, en este caso **regla1_energoS3_nico**. En la instrucción SQL escribimos **SELECT FROM ‘objeto_energo_1_nico/#’**

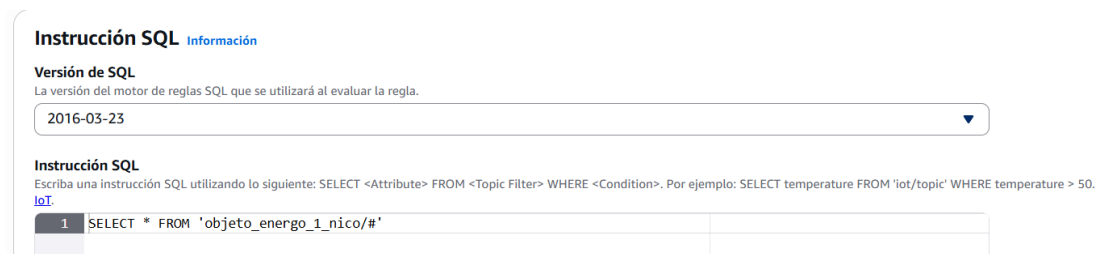


Figura 28: Instrucción SQL de la regla

Hacemos click en crear acción y seleccionamos “Amazon S3 bucket” (Almacenar mensajes en un bucket de S3 existente). Ahora buscamos nuestro bucket creado previamente **bucketenergo1nico**.

En clave, usamos la siguiente sintaxis para que los archivos se organicen automáticamente por año, mes, día y hora de recepción.

```
${topic()}/year=${timestamp("yyyy")}/month=${timestamp("MM")}/day=${timestamp("dd")}/hour=${timestamp("HH")}/${timestamp()}-data.json
```

Acción 1
S3 bucket
Almacenar mensajes en un bucket de Simple Storage Service (Amazon S3) existente

Nombre del bucket [Información](#)
URI de S3
s3://bucketenergo1nico

Clave
La clave de S3 de este mensaje.
\${topic()}/year=\${timestamp("yyyy")}/month=\${timestamp("MM")}/day=\${timestamp("dd")}/hour=\${timestamp("HH")}/\${timestamp()}-data.json

Eliminar

Ver

Explorar S3

Figura 29: Elección del bucket y clave de S3

Por último nos pide elegir un rol para conceder a AWS IoT acceso al punto de conexión. Necesitamos permisos para crear un rol ya que ningún rol predefinido tiene el permiso `s3:PutObject` y un rol que permite que una regla almacene objetos en S3 necesita tener el dicho permiso.

No podemos probar esa última parte al completo y comprobar que los objetos se almacenan en el bucket debido a una restricción de permisos con los laboratorios de AWS.

Crear un rol

⊗ ▶ Acceso denegado a iam:CreateRole

Nombre de rol
energo_rol

Ingrese un nombre de rol único que contenga caracteres alfanuméricos, guiones y guiones bajos. Un nombre de rol no puede contener espacios.

Cancelar Crear

Figura 30: Acceso denegado a la creación de roles

Extensión: Conexión API REST para precios de proveedores

En este apartado se pide diseñar un flujo en Node-RED donde la ingesta se haga a través de la conexión al API de la página <https://www.ree.es/es/datos/apidatos>, recoger los precios de diferentes proveedores para hacer comparativas y enviarlos a otro

topic de la nube de AWS.

Puesto que la conexión API para extraer los diferentes precios falla, pues al cargar la url `https://apidatos.ree.es/es/datos/mercados/precios-mercado-tiempo-real?start_date=2025-10-01T00:00&end_date=2025-10-01T23:59&time_trunc=hour` encontramos el error de la Figura 31, vamos a realizar el mismo ejercicio pero solicitando el widget general mensual IRE para el año 2018 con límite geográfico peninsular español.

Oops! An Error Occurred

The server returned a "500 Internal Server Error".

Something is broken. Please let us know what you were doing when this error occurred. We will fix it as soon as possible. Sorry for any inconvenience caused.

Figura 31: Error al conectar a través de la API

Creamos el siguiente flujo:

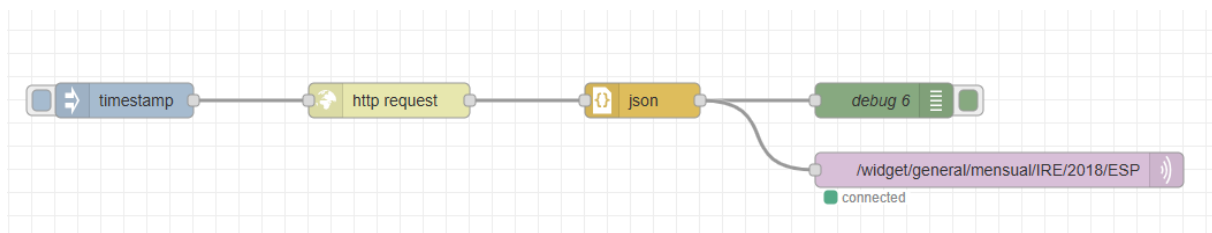


Figura 32: Flujo 4. Extracción de datos de REE a través de la API

1. Nodo inject para iniciar el flujo.
2. El nodo http request realiza un GET de la url `https://apidatos.ree.es/es/datos/demanda/ire-general?start_date=2018-01-01T00:00&end_date=2018-12-31T23:59&time_trunc=month&geo_trunc=electric_system&geo_limit=peninsular&geo_ids=8741`, extraída de la página `https://www.ree.es/es/datos/apidatos`.
3. Con el módulo json configurado por defecto, convertimos texto JSON a objeto JavaScript

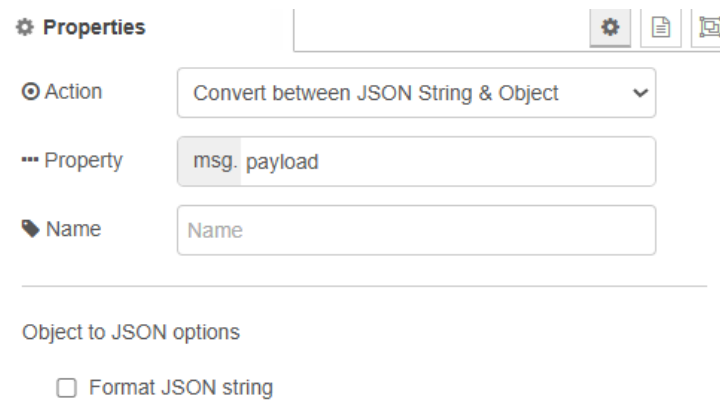


Figura 33: Configuración del módulo json

4. El nodo debug nos permite visualizar el output deseado

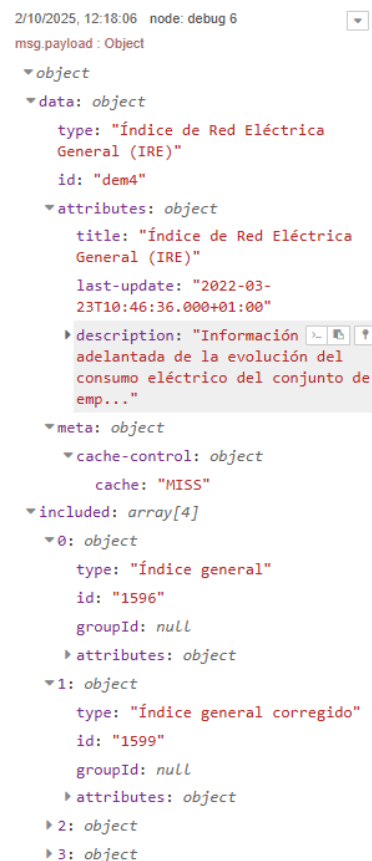


Figura 34: Output del módulo debug 6

5. La configuración del mqtt out es la siguiente

Properties

Server: a1s639ohtzloz-ats.iot.us-east-1

Topic: /widget/general/mensual/IRE/2018/ESP

QoS: 1

Name: Name

Figura 35: Configuración del módulo *mqtt out*

donde el servidor es exactamente el mismo que para el resto de nodos que realizan conexión con AWS y el topic elegido es **/widget/general/mensual/IRE/2018/ESP** donde comprobaremos que están llegando los mensajes.

Por último comprobamos suscribiéndonos al topic **/widget/general/mensual/IRE/2018/ESP** en AWS, que los mensajes están siendo recibidos.

Suscripciones

/widget/general/mensual/IRE/2018/ESP

Carga del mensaje

```
{
  "message": "Hello from AWS IoT console"
}
```

Configuración adicional

Publicar

/widget/general/mensual/IRE/2018/ESP

October 02, 2025, 12:32:29 (UTC+0200)

```
{
  "data": {
    "type": "Índice de Red Eléctrica General (IRE)",
    "id": "demo",
    "attributes": {
      "title": "Índice de Red Eléctrica General (IRE)",
      "last-update": "2022-03-23T18:46:36.000+01:00",
      "description": "Información adelantada de la evolución del consumo eléctrico del conjunto de empresas que tienen una potencia contratada superior a 450 kW. Índice 2018-100 y % de crecimiento sobre el mismo periodo del año anterior"
    }
  },
  "meta": {
    "cache-control": {
      "cache": "no",
      "expires": "2025-11-01T10:18:05"
    }
  },
  "included": [
    {
      "type": "Índice general",
      "id": "general"
    }
  ]
}
```

Figura 36: Suscripción al topic **/widget/general/mensual/IRE/2018/ESP**